

Step 3 — Retry and Fallback for the Drafting Call — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below are run from inside `project_13_langchain_patterns/`, continuing from Step 2's `handlers.py`.

Basic Version

Approach 1 — retry only, no fallback yet

```
# drafting.py
from langchain_openai import ChatOpenAI
from openai import RateLimitError, APITimeoutError, APIConnectionError

def build_resilient_model():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
    return model.with_retry(
        retry_if_exception_type=(RateLimitError, APITimeoutError, APIConnectionError),
        stop_after_attempt=3,
    )

resilient_model = build_resilient_model()
print(resilient_model.invoke("Say hello in one short sentence."))
```

Expected output:

```
content='Hello! How can I help you today?' ...
```

On a normal call this behaves exactly like a plain `ChatOpenAI` call — you only notice the difference when a transient error actually happens, which is the whole point of testing it on purpose next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real fallback chain, and `.with_fallbacks()`

```
# fallback.py
# Design decision, written down on purpose: a canned reply is an acceptable
# stand-in for general and technical tickets while the primary drafting
# call is down. For billing tickets, the canned reply still goes out (a
# customer shouldn't be left with total silence), but it's paired with
# needs_human_review=True in Step 4, so a person confirms the real charge
# issue before it's ever treated as resolved.

from langchain_core.runnables import RunnableLambda

def canned_response(inputs):
    return (
        "Thanks for reaching out. We've received your message and a "
        "member of our support team will follow up with you shortly."
    )

def build_fallback_chain():
    return RunnableLambda(canned_response)
```

```
# handlers.py (updated)
from langchain_core.output_parsers import StrOutputParser
from drafting import build_resilient_model
from fallback import build_fallback_chain
from research import build_research_chain

resilient_model = build_resilient_model()
fallback_chain = build_fallback_chain()
research_chain = build_research_chain()

billing_chain = (
    research_chain | billing_prompt | resilient_model | StrOutputParser()
).with_fallbacks([fallback_chain])

technical_chain = (
```

```
research_chain | technical_prompt | resilient_model | StrOutputParser()  
.with_fallbacks([fallback_chain])  
  
general_chain = (  
    general_prompt | resilient_model | StrOutputParser()  
).with_fallbacks([fallback_chain])
```

Expected behavior on a normal, working call: identical output to Step 2 — the fallback never runs, because the primary chain never raised.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Approach 2 — proving the fallback actually fires, on purpose

```
# test_pipeline.py (excerpt)
from langchain_core.runnables import RunnableLambda
from fallback import build_fallback_chain

def always_fails(inputs):
    raise TimeoutError("simulated transient failure for testing")

broken_primary = RunnableLambda(always_fails)
fallback_chain = build_fallback_chain()
safe_chain = broken_primary.with_fallbacks([fallback_chain])

result = safe_chain.invoke({"ticket": "test"})
print(result)
```

Expected output:

```
Thanks for reaching out. We've received your message and a member of our support
```

The primary chain always raises, so `.with_fallbacks()` always runs the fallback instead — proof the wiring genuinely works, not just that it compiles.

Difference from Basic: Basic wraps the model with retry alone — useful, but if all 3 retry attempts still fail, the chain still crashes with no fallback. Intermediate adds a genuine second path via `.with_fallbacks()`, and Approach 2 proves, with a deliberately broken chain, that the fallback path really does run when it's supposed to.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a cheaper-model fallback instead of a canned template

```
# fallback.py (alternative: a real, cheaper model instead of a fixed string)
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

FALLBACK_PROMPT = ChatPromptTemplate.from_template(
    "Write a brief, polite acknowledgment for this support ticket. "
    "Do not promise specific outcomes. Ticket: {ticket}"
)

def build_fallback_chain():
    fallback_model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return FALLBACK_PROMPT | fallback_model | StrOutputParser()
```

Expected behavior: if the primary (presumably pricier or slower) model call fails, this fallback still makes a real model call, just to a smaller, cheaper, hopefully more available model — a genuinely different degraded path than a fixed canned string, at the cost of still depending on *some* model call succeeding.

Approach 2 — refusing to send a degraded answer for a high-stakes category

```
# handlers.py -- treating billing differently on purpose
class NeedsHumanEscalation(Exception):
    """Raised instead of silently sending a degraded billing reply."""

def billing_fallback(inputs):
    raise NeedsHumanEscalation(
        "Primary drafting failed for a billing ticket -- escalating to a "
        "human instead of sending a canned reply about a real charge."
    )

from langchain_core.runnables import RunnableLambda
```

```

billing_chain = (
    research_chain | billing_prompt | resilient_model | StrOutputParser()
).with_fallbacks([RunnableLambda(billing_fallback)])

```

Expected behavior: on the billing path, once retries are exhausted, this raises a clear, named exception instead of quietly sending a reply about a real financial issue — the caller (`main.py` , or whatever's calling this pipeline in production) is expected to catch `NeedsHumanEscalation` and route the ticket to a person directly, rather than treat it as "handled."

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's single canned-reply fallback treats every category the same way. Approach 1 shows a genuinely different *quality* of degraded answer (a real, cheaper model call) instead of a fixed string. Approach 2 shows the other real option: for a category where a wrong degraded answer is worse than no answer, "fail loudly and escalate" can be the more honest fallback than any generated text at all — a fallback doesn't have to produce a response; it can produce a clear failure that a human is supposed to see.

Which one should you actually write? Whichever one you can defend out loud, per category, and it's fine for different categories to get different answers. A reasonable real-world split: general and technical tickets get Intermediate's canned-reply fallback (low stakes, and a slightly generic reply costs little); billing tickets get Approach 2's loud escalation, because a customer's real money is involved and a wrong guess there is worse than a short delay. What's not reasonable is picking one fallback strategy for the whole pipeline without ever asking whether every category actually deserves the same answer to "what's acceptable when things go wrong."